

Laboratorio 0

Entorno de trabajo y primer pull request

Se corre en dos miércoles: el 19 (Partes 0–2: tu entorno y tu primer programa) y el 26 (Partes 3–4: Git, GitHub y tu primera entrega, que ya cuenta como la P1). Cuatro puntos de control.

TU SISTEMA

Windows

macOS

Linux

Antes de empezar: elige tu sistema arriba

Los comandos cambian según el sistema. Toda la guía se adapta a lo que elijas. Si te equivocas, cámbialo cuando quieras.

Parte 0 · En casa, antes del miércoles

Esto **no** se hace en el laboratorio. Se hace antes, y el miércoles solo verificamos. Si algo no instala, avísame el martes, no el miércoles a las 15:05.

0.1 · Instalar el JDK 21

El JDK es el kit que compila y ejecuta Java. Vamos con la versión **21**, que es LTS (soporte extendido) y la que usa la industria hoy.

Si tienes [Homebrew](#):

```
brew install --cask temurin@21
```

Si no tienes Homebrew, descarga el `.pkg` desde [adoptium.net](#) (elige **aarch64** si tu Mac es M1/M2/M3/M4, o **x64** si es Intel) y sigue el asistente.

0.2 • Instalar Git

Lo más simple, en la Terminal:

```
xcode-select --install
```

Eso instala las herramientas de línea de comandos de Apple, que incluyen Git. Si prefieres la versión más nueva: `brew install git`.

0.3 • Instalar el IDE

Vamos con **IntelliJ IDEA**. Desde finales de 2025 es **un solo producto**: la "Community Edition" que vas a ver mencionada en tutoriales viejos ya no existe como descarga aparte. El instalador es uno, la **capa gratuita viene activada por defecto** y no pide cuenta ni licencia. Las funciones de paga quedan apagadas y no las vamos a necesitar.

```
brew install --cask intellij-idea
```

O desde jetbrains.com/idea/download. Hay un solo botón de descarga; no busques "Community".

0.4 • Crear tu cuenta de GitHub

En github.com/signup. Dos cosas importantes:

- Usa un **nombre de usuario que puedas enseñar en una entrevista**. Este repositorio va a ser lo primero que tengas en tu portafolio.
- Usa un correo al que **tengas acceso siempre**, no el institucional que pierdes al egresar.

Entregable de la Parte 0

Llega el miércoles con las cuatro cosas instaladas y tu usuario de GitHub anotado. La Parte 1 tarda cinco minutos si hiciste esto, y toda la sesión si no.

Parte 1 • Verificar el entorno

Abre tu terminal.

Terminal (Cmd + Espacio → escribe "Terminal").

Corre estos tres comandos, uno por uno:

```
java -version
javac -version
git --version
```

SALIDA ESPERADA, ALGO ASÍ

```
openjdk version "21.0.4" 2024-07-16 LTS
javac 21.0.4
git version 2.45.2
```

Los números pueden variar. Lo que importa: que **diga 21** en las dos primeras, y que ninguna diga "command not found".

Si algo falla

LO QUE VES

QUÉ PASA Y CÓMO SE ARREGLA

`java: command not found`

El JDK no está instalado, o no está en el **PATH**. Qué es eso y cómo se arregla a mano, paso a paso: abajo, en "**El PATH, explicado desde cero**".

`java` sí funciona
pero `javac` no

Instalaste un **JRE** (solo ejecuta) en vez de un **JDK** (compila y ejecuta).
Instala el JDK.

Dice otra versión, por ejemplo 17 u 8

Tienes varias versiones y el sistema está tomando otra. Avísame en el laboratorio, se resuelve en dos minutos.

macOS: "no se puede abrir porque el desarrollador no está verificado"

Ve a **Ajustes del Sistema → Privacidad y seguridad**, baja hasta el aviso y dale "**Abrir de todos modos**".

El PATH, explicado desde cero

Cuando escribes `java` y das Enter, la terminal no adivina dónde está el programa: lo busca en una **lista de carpetas, en orden**, y ejecuta el primero que encuentre. Esa lista se llama el

PATH. Si la carpeta donde quedó Java no está en la lista, la terminal contesta `command not found` aunque Java sí esté instalado. No hay nada roto: la terminal solo no sabe dónde buscar.

Puedes ver tu lista ahora mismo. Este comando la imprime, una carpeta por renglón, y es el mismo en los tres sistemas:

```
echo $PATH | tr ':' '\n'
```

Las dos reglas que explican casi todos los sustos

1 • Los instaladores agregan su carpeta al PATH por ti; por eso normalmente no lo tocas nunca. **2** • La terminal lee el PATH **una sola vez, al abrirse**. Después de cualquier cambio, la ventana que ya estaba abierta no se entera: cierra y abre una nueva.

En macOS **no tienes que tocar el PATH para Java**: el instalador deja el JDK en una carpeta que el sistema ya vigila, y el `java` de `/usr/bin` lo encuentra solo. Si `java -version` no responde, pregúntale al sistema qué JDKs está viendo:

```
/usr/libexec/java_home -V
```

Si tu JDK 21 no aparece en esa lista, la instalación no terminó: vuelve a correr el `.pkg`. Si sí aparece y aun así falla, avísame y lo resolvemos en el laboratorio.

1 Punto de control 1

Los tres comandos responden y las versiones de Java dicen **21**. Marca CP1 en el tablero.

Parte 2 • Tu primer programa, sin IDE

Vamos a compilar y ejecutar a mano, con dos comandos. Después usaremos el IDE todo el semestre, pero **primero hay que ver qué hace el compilador**. El IDE hace esto mismo escondido detrás de un botón verde.

2.1 · Crear la carpeta y el archivo

```
# crea una carpeta para el curso y entra en ella
mkdir -p ~/mpoo
cd ~/mpoo
```

Ahora crea un archivo llamado `HolaMundo.java` con este contenido. Puedes usar el Bloc de notas, TextEdit, nano, o el editor que quieras:

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola, mundo");
    }
}
```

El nombre del archivo importa

Tiene que llamarse **exactamente** `HolaMundo.java`, con la H y la M mayúsculas. En Java, el archivo **debe llamarse igual que la clase pública** que contiene. Es la primera regla del lenguaje que vas a romper sin querer, y hoy la rompemos a propósito para verla.

2.2 · Compilar

```
javac HolaMundo.java
```

Si todo salió bien, **no imprime nada**. En Unix el silencio es señal de éxito. Ahora mira qué apareció:

```
ls
```

SALIDA ESPERADA

```
HolaMundo.class    HolaMundo.java
```

Ese `HolaMundo.class` no lo escribiste tú: **lo generó el compilador**. No es código Java ni es un ejecutable de tu sistema: es **bytecode**, un lenguaje intermedio que entiende la máquina virtual de Java. Es la razón por la que el mismo archivo corre igual en Windows, Mac y Linux.

2.3 · Ejecutar

```
java HolaMundo
```

SALIDA ESPERADA

Hola, mundo

Fíjate en el detalle: al compilar escribiste `HolaMundo.java` con extensión, y al ejecutar escribiste `HolaMundo` **sin extensión**. No le pasas un archivo: le pasas el **nombre de la clase** que quieres correr.

2.4 · Rómpelo a propósito

Quítale el punto y coma a la línea del `println`, guarda y vuelve a compilar:

```
javac HolaMundo.java
```

SALIDA ESPERADA

```
HolaMundo.java:3: error: ';' expected
    System.out.println("Hola, mundo")
                        ^
```

1 error

Lee el mensaje completo: te dice **archivo, línea, qué esperaba y dónde**. Vas a leer cientos de estos este semestre. El que aprende a leerlos avanza el triple. Vuelve a poner el punto y coma y compila otra vez.

2.5 · Lo mismo, ahora en el IDE

1. Abre IntelliJ IDEA → **New Project**.
2. Nómbralo `hola-ide`, elige **Java** y como SDK el **21** que instalaste.
3. En `src`, clic derecho → **New** → **Java Class** → nómbrala `HolaMundo`.

4. Escribe el mismo `main` y dale al **triángulo verde**.

Mira la ventana de abajo cuando corre: ahí está **el mismo** `javac` y **el mismo** `java` que acabas de escribir a mano. El IDE no hace magia, escribe los comandos por ti.

2 Punto de control 2

Hola, mundo corriendo **desde la terminal** y también desde IntelliJ. Marca CP2.

Parte 3 · Git: dónde va a vivir tu trabajo

3.1 · Cómo funciona el repositorio del curso

Todo el material del curso vive en un repositorio público de GitHub. Yo subo ahí las guías y el código de apoyo; ustedes suben ahí sus entregas.

```
mpoo-2027-1/
├─ guías/           # las guías de laboratorio (las subo yo)
├─ apoyo/           # código de ejemplo de cada tema (lo subo yo)
└─ entregas/        # aquí van sus prácticas
    ├─ ramirez_ana/
    │   ├─ lab00/
    │   └─ p01/
    └─ cruz_luis/
        └─ lab00/
```

Cada quien toca solo su carpeta. Por eso 27 personas pueden trabajar en el mismo repositorio sin pisarse.

3.2 · Presentarte con Git

Git firma cada cambio con tu nombre. Configúralo una sola vez en tu vida:

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "el-correo-de-tu-cuenta-de-github@ejemplo."
```

Usa el mismo correo de tu cuenta de GitHub

Si pones otro, tus commits aparecen como de un desconocido y no se te acreditan. Y esa acreditación es justamente la evidencia de trabajo en equipo que se califica en el proyecto final.

3.3 · Vuélvete colaborador

Todos trabajamos **en el mismo repositorio**, como un equipo en una empresa. Pero para poder subir algo, primero tengo que darte acceso:

1. **Mándame tu usuario de GitHub** por el chat del grupo, en una línea: `usuario: fulanito123`.
2. Te va a llegar una **invitación**: revisa tu correo o `github.com/notifications`.
3. Dale **Accept invitation**. Sin este paso no vas a poder subir nada.

¿Y no puedo romper algo?

No. La rama principal, `main`, está **protegida**: nadie escribe directo en ella, ni por accidente. Todo entra por **pull request** que yo apruebo, y tú siempre trabajas en una rama tuya.

3.4 · Bajarlo a tu computadora (clone)

Se clona **el repositorio del curso**, el mismo para todos:

```
cd ~/mpoo
git clone https://github.com/OscarRuiz21/mpoo-2027-1.git
cd mpoo-2027-1
```

Comprueba que estás parado en el lugar correcto:

```
git status
```

SALIDA ESPERADA


```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

"**nothing to commit, working tree clean**" quiere decir: no has cambiado nada todavía. Vas a ver ese mensaje mil veces.

3 Punto de control 3

Git configurado con tu nombre, invitación aceptada, repositorio clonado y `git status` respondiendo. Marca CP3.

Parte 4 · Tu primera entrega: la P1

4.1 · Crea tu rama

Regla de oro del curso: **nunca trabajas en `main`**. Cada práctica vive en una rama con el número de la práctica y tu apellido:

```
# parado en main, con lo último
git checkout main
git pull

# tu rama de la práctica 1
git checkout -b p01-apellido
```

Una **rama** es una línea de trabajo aparte: lo que hagas ahí no toca lo de nadie. En la sesión 6 la vamos a entender a fondo; hoy basta con saber usarla.

4.2 · Tu carpeta y tu programa

La carpeta usa **tu primer apellido y tu primer nombre, en minúsculas y sin acentos, unidos por un guion bajo**: así dos personas con el mismo apellido no chocan. Si te llamas Ana Ramírez, tu carpeta es `ramirez_ana`. La mía sería `ruiz_oscar`.

```
mkdir -p entregas/apellido_nombre/p01
cp ~/mpoo/HolaMundo.java entregas/apellido_nombre/p01/
```

Y ahora **personalízalo**, porque la P1 pide un programa tuyo, no el de la guía. Edítalo para que use **dos variables y las concatene**:

```
public class HolaMundo {
    public static void main(String[] args) {
        String nombre = "Ana";        // el tuyo
        String apellido = "Ramírez"; // el tuyo
        System.out.println("Hola, soy " + nombre + " " + apellido);
    }
}
```

Ese `+` entre cadenas se llama **concatenación** y es tu primer contacto con `String`.
Compíllalo y córrelo para comprobar que imprime tu nombre.

4.3 · Guardar: los tres comandos de siempre

```
# 1. ¿qué cambió?
git status

# 2. preparo lo que quiero guardar
git add entregas/apellido_nombre/p01

# 3. lo guardo con un mensaje que explique qué hice
git commit -m "P01: entorno y hola mundo con mi nombre"
```

SALIDA ESPERADA DEL COMMIT

```
[p01-ramirez a3f9c21] P01: entorno y hola mundo con mi nombre
1 file changed, 7 insertions(+)
```

Ese `a3f9c21` es el identificador único de tu cambio. Tu trabajo ya está guardado **en tu computadora**. Falta subirlo.

4.4 · Subir tu rama (push)

```
git push -u origin p01-apellido
```

El `-u origin p01-apellido` solo se escribe la **primera vez** que subes una rama nueva; después basta `git push`.

La primera vez te va a pedir usuario y contraseña. **La contraseña de GitHub ya no funciona ahí:** hay que usar un **token**. Sácalo así:

1. GitHub → tu foto arriba a la derecha → **Settings**.
2. Hasta abajo: **Developer settings** → **Personal access tokens** → **Tokens (classic)**.
3. **Generate new token (classic)**. Nómbralo `mpoo`, marca la casilla **repo**, y généralo.
4. **Cópialo ya:** no lo vuelves a ver. Pégallo donde te pide la contraseña.

Guarda el token

Pégalo en tus notas. Lo vas a necesitar cada vez que subas algo desde una máquina nueva, y los laboratorios de la Facultad se borran al reiniciar.

4.5 · Abrir tu pull request

Un **pull request** (PR) es la forma profesional de decir "aquí está mi trabajo, revísenlo". Así se entrega en la industria, y así se entrega aquí:

1. Entra a `github.com/OscarRuiz21/mpoo-2027-1`. Arriba aparece un aviso amarillo con tu rama y el botón **Compare & pull request**. Dale.
2. Título: `P01 · Tu Apellido`. En la descripción, una línea de qué hiciste.
3. Botón verde **Create pull request**. Listo.

Yo lo reviso y lo integro a `main`. **Tu entrega es el PR:** la hora que cuenta es la de tu último push a la rama.

4 Punto de control 4

Tu PR abierto y visible en la pestaña **Pull requests** del repositorio del curso, con tu `HolaMundo.java` personalizado adentro. Marca CP4 y avísame para revisarlo.

Cómo les subo el material nuevo

Cada semana subo la guía y el código de apoyo a `main`. Para traerlo, párate en `main` y jala:

```
git checkout main
git pull
```

Hazlo **al inicio de cada laboratorio**, antes de crear tu rama nueva. Cinco segundos y te evita trabajar con la guía vieja.

Qué entregas y cuándo

QUÉ	CUÁNDO
P1 • Entorno y lenguaje de programación: tu PR con <code>HolaMundo.java</code> personalizado (nombre y apellido concatenados)	Hoy, en clase
P2 • Abstracción y modelado: se presenta la próxima sesión de laboratorio	Miércoles 2 de septiembre

La regla de puntualidad y la política de uso de IA están en el encuadre, en el repositorio. Dos recordatorios de hoy: **en el laboratorio se trabaja sin IA**, y **si la usaste en casa, se declara en una línea al inicio del archivo**.

Los pasos: cómo se entrega, siempre

Esta es la rutina completa. **La vas a repetir doce veces este semestre**, una por práctica. Tómale captura.

Rutina de entrega

De la práctica 1 a la 12 · siempre igual



Antes de empezar: trae lo nuevo

```
cd ~/mpoo/mpoo-2027-1
```

```
git checkout main
```

```
git pull
```

Baja las guías y el código de apoyo que subí esta semana. **Cinco segundos y te evita trabajar con la guía vieja.**

1

```
git checkout -b pNN-apellido
```

Tu rama de la práctica. Nunca trabajas en `main`: está protegida y no te va a dejar.

Ejemplo: `p03-ramirez`.

2

Trabaja

Tu código va en `entregas/apellido_nombre/pNN/`. Nunca toques la carpeta de otro ni `guias/` ni `apoyo/`.

3

```
git status
```

¿Qué cambió? Míralo siempre antes de guardar. Si aparece un archivo que no esperabas, ahí lo cachas.

4

```
git add entregas/apellido_nombre/pNN
```

Preparo lo que quiero guardar. Nombra tu carpeta, no uses `git add .`: eso agarra todo lo que haya, incluida basura del IDE.

5

```
git commit -m "P03: arreglos y arreglos multidimensionales"
```

Lo guardo con un mensaje que explique qué hice. Empieza con el número de práctica. Un mensaje como "cambios" o "asdf" no le sirve a nadie, empezando por ti en noviembre.

6

```
git push -u origin pNN-apellido
```

Subo mi rama a GitHub. El `-u origin ...` solo la primera vez; después basta

`git push`. Sin push no hay nada.

7

Abre tu PR y comprueba

En el repo del curso: **Compare & pull request** → título `PNN · Apellido` → **Create**.

Tu entrega es el PR, y la hora que cuenta es la de tu último push.

LO QUE PASA

QUÉ SIGNIFICA Y QUÉ HACES

`nothing to commit`
después del `add`

No guardaste el archivo en el editor, o lo pusiste en otra carpeta.
Corre `git status` y revisa la ruta.

Hiciste `commit` pero
no aparece en GitHub

Falta el `push`. El commit guarda en tu computadora; el push lo
sube. Es el error número uno.

`protected branch` al
hacer `push`

Commiteaste en `main` sin querer. No pasa nada: corre `git
checkout -b pNN-apellido`; la rama nueva se lleva tus commits,
y desde ahí pusheas normal.

Olvidaste el `-u origin`
`...` en el primer push

Git te lo dice con el comando exacto en pantalla. Cópialo y córrelo:
es eso.

Te pide contraseña y la
rechaza

La contraseña de GitHub no sirve para `push`. Usa tu **token** (Parte
4.4). Si lo perdiste, genera otro.

Si te atorras y no sabes qué hiciste

No borres la carpeta ni vuelvas a clonar. Corre `git status`, tómale captura y
mándamela. Casi siempre son dos comandos para salir, y aprender a leer `git
status` vale más que cualquier atajo.